

# Digital Wallet & Money Transfer System

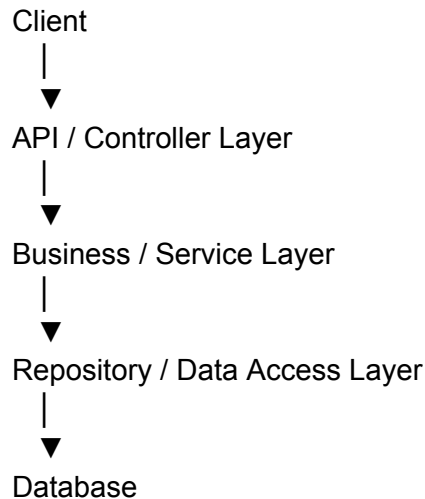
## 1. Project Objective

Build a backend system where users can create wallets, deposit money, withdraw money, and transfer money to other users.

The main objective is **not just to build CRUD APIs**.

The interns must design the application so that all important business rules are handled in a dedicated **Business/Service Layer**.

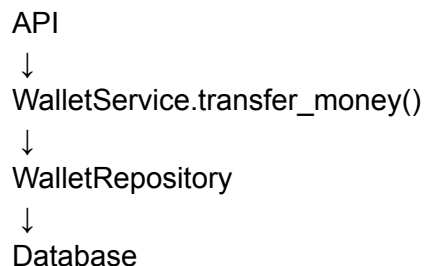
### Required architecture



The API layer should **not** contain business decisions such as:

```
if balance < amount:
    reject_transaction()
```

Instead:



---

## 2. User Roles

The system should have three roles.

### Customer

Can:

- Create wallet
- View wallet
- Deposit money
- Withdraw money
- Transfer money
- View transactions
- View transaction details

### Admin

Can:

- View users
- View wallets
- Freeze/unfreeze wallets
- View all transactions
- Investigate failed transactions
- Adjust wallet balance through a controlled process

### System

The system itself should handle:

- Transaction processing
  - Balance updates
  - Transaction status
  - Daily limits
  - Duplicate transaction prevention
  - Transaction history
-

### 3. Main Entities

Interns should design their own database schema, but they should have at least these entities:

User  
Wallet  
Transaction  
WalletTransaction

A possible design:

User  
----  
id  
name  
email  
password\_hash  
status  
created\_at  
updated\_at  
Wallet  
-----  
id  
user\_id  
balance  
currency  
status  
created\_at  
updated\_at  
Transaction  
-----  
id  
reference  
type  
status  
amount  
currency  
source\_wallet\_id  
destination\_wallet\_id  
description  
created\_at  
updated\_at

Possible transaction types:

DEPOSIT  
WITHDRAWAL  
TRANSFER

Possible statuses:

PENDING  
COMPLETED  
FAILED  
CANCELLED

---

## 4. Wallet Rules

Every user can have **only one active wallet per currency**.

For example:

User 1  
├── USD Wallet  
└── PKR Wallet

But they cannot have:

User 1  
├── PKR Wallet #1  
└── PKR Wallet #2

unless the business requirements explicitly allow it.

### Wallet statuses

ACTIVE  
FROZEN  
CLOSED

A frozen wallet cannot perform financial transactions.

For example:

Frozen Wallet  
├── Deposit ✗  
├── Withdrawal ✗  
└── Transfer ✗

---

## 5. Deposit Money

Create an API such as:

POST /api/wallet/deposit/

Request:

```
{  
  "amount": 5000  
}
```

### Business rules

The server must verify:

1. Wallet exists.
2. Wallet belongs to the authenticated user.
3. Wallet is active.
4. Amount is greater than zero.
5. Amount does not exceed the maximum deposit limit.
6. Currency is valid.
7. Transaction is not duplicated.

If everything is valid:

Old Balance

↓

5000

Deposit

↓

2000

New Balance

↓

7000

A transaction record must also be created.

---

## 6. Withdrawal

API:

POST /api/wallet/withdraw/

Request:

```
{  
  "amount": 2000  
}
```

### Business rules

The service layer should check:

Wallet exists



Wallet ACTIVE?



Amount > 0?



Daily withdrawal limit?



Sufficient balance?



Process withdrawal

Example:

Balance = 10,000

Withdrawal = 3,000

New Balance = 7,000

But:

Balance = 10,000

Withdrawal = 15,000

→ Reject

The balance must **never become negative**.

---

## 7. Money Transfer

This should be the most important part of the assignment.

API:

POST /api/wallet/transfer/

Request:

```
{  
  "recipient_wallet_id": 25,  
  "amount": 3000,  
  "description": "Payment for services"  
}
```

Suppose:

Alice

Balance = 10,000

Bob

Balance = 5,000

Alice transfers:

3,000

After successful transaction:

Alice = 7,000

Bob = 8,000

---

## 8. Transfer Business Rules

The service layer must validate all of these.

### Rule 1 — Sender wallet exists

Sender wallet not found

→ Reject

### **Rule 2 — Receiver wallet exists**

Receiver wallet not found

→ Reject

### **Rule 3 — Sender cannot transfer to themselves**

Sender = Wallet 10

Receiver = Wallet 10

→ Reject

### **Rule 4 — Both wallets must be active**

Sender = ACTIVE

Receiver = FROZEN

→ Reject

### **Rule 5 — Amount must be positive**

amount  $\leq 0$

→ Reject

### **Rule 6 — Sender must have sufficient balance**

Balance = 5,000

Transfer = 7,000

→ Reject

### **Rule 7 — Daily transfer limit**

For example:

Daily transfer limit = 100,000

If the user has already transferred:

95,000

they cannot transfer another:

10,000

because:



95,000 + 10,000 > 100,000

## **Rule 8 — Transaction must be atomic**

This is extremely important.

You must never end up with:

Alice = 7,000

Bob = 5,000

after Alice attempted to transfer 3,000.

Either:

Alice -3000

Bob +3000

Transaction COMPLETED

or:

Alice unchanged

Bob unchanged

Transaction FAILED

Never a partial update.

---

## **9. Database Transaction**

Interns should understand database transactions.

Conceptually:

with database.transaction():

deduct\_money(sender)

add\_money(receiver)

create\_transaction()

If something fails:

Rollback

So:

Deduct sender



Something fails



Rollback



Sender's balance restored

This is one of the most important concepts in this assignment.

---

## 10. Duplicate Transaction Prevention

Introduce an **idempotency key**.

For example:

POST /api/wallet/transfer/  
Idempotency-Key: abc123

Request:

```
{  
  "recipient_wallet_id": 25,  
  "amount": 3000  
}
```

Suppose the client sends the request twice because of a network problem.

Without idempotency:

Request 1 → Transfer 3000

Request 2 → Transfer 3000

Alice loses 6000 

With idempotency:

Request 1



Transaction abc123



Completed

Request 2



abc123 already processed



Return previous result

This is a **very good advanced requirement** for interns.

---

## 11. Transaction History

API:

GET /api/wallet/transactions/

Response:

```
[
  {
    "reference": "TXN-10001",
    "type": "TRANSFER",
    "amount": 3000,
    "status": "COMPLETED",
    "created_at": "2026-08-11T10:00:00"
  }
]
```

Support:

- Pagination
  - Filtering by transaction type
  - Filtering by status
  - Date range
  - Amount range
  - Search by transaction reference
-

## 12. Transaction Reference

Every transaction should have a unique reference:

TXN-20260811-000001

TXN-20260811-000002

TXN-20260811-000003

The reference must be unique.

Interns should think about whether the reference should be generated:

Application side

or

Database side

and how to guarantee uniqueness under concurrent requests.

---

## 13. Daily Limits

Introduce configurable limits.

For example:

Maximum single withdrawal:

50,000

Maximum daily withdrawal:

100,000

Maximum single transfer:

100,000

Maximum daily transfer:

500,000

These should **not be hard-coded throughout the codebase**.

Instead, create something like:

WalletConfiguration

or a configuration/service mechanism.

Then:

```
if amount > config.max_transfer_amount:  
    raise BusinessRuleError(...)
```

---

## 14. Wallet Freeze

Admin can freeze a wallet.

API:

POST /api/admin/wallets/{id}/freeze/

Once frozen:

|            |   |
|------------|---|
| Deposit    | ✗ |
| Withdrawal | ✗ |
| Transfer   | ✗ |

But:

|                   |   |
|-------------------|---|
| View balance      | ✓ |
| View transactions | ✓ |

Admin can later:

POST /api/admin/wallets/{id}/unfreeze/

---

## 15. Balance Adjustment

Give admins the ability to correct a wallet balance.

But don't allow:

PUT /wallet/10/

with:

```
{
  "balance": 500000
}
```

That would be terrible design.

Instead create a controlled operation:

POST /api/admin/wallets/{id}/adjust-balance/

Request:

```
{
  "amount": 500,
  "reason": "Manual correction after payment investigation"
}
```

This should create an auditable transaction.

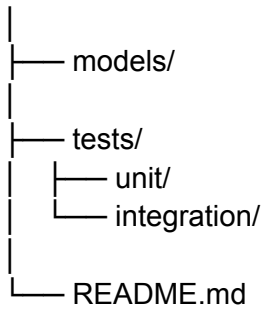
---

## 16. Service Layer

This is where you should focus the interns.

Possible structure:

```
project/
|
|-- api/
|   |-- views/
|   |-- serializers/
|   |-- urls/
|
|-- services/
|   |-- wallet_service.py
|   |-- transfer_service.py
|   |-- transaction_service.py
|   |-- admin_wallet_service.py
|
|-- repositories/
|   |-- wallet_repository.py
|   |-- transaction_repository.py
|   |-- user_repository.py
```



The exact structure is up to them, but the separation of responsibilities should be clear.

---

## 17. Example Service

A simplified conceptual implementation:

class WalletService:

```
    def transfer(
        self,
        sender_wallet_id,
        receiver_wallet_id,
        amount,
        idempotency_key
    ):
        self.validate_amount(amount)

        transaction = self.get_existing_transaction(
            idempotency_key
        )

        if transaction:
            return transaction

        sender = self.get_wallet(sender_wallet_id)
        receiver = self.get_wallet(receiver_wallet_id)

        self.validate_wallet(sender)
        self.validate_wallet(receiver)

        self.validate_different_wallets(
            sender,
```

```

        receiver
    )

    self.validate_balance(
        sender,
        amount
    )

    self.validate_daily_limit(
        sender,
        amount
    )

    # Database transaction
    with self.transaction():

        self.debit(sender, amount)
        self.credit(receiver, amount)

        transaction = self.create_transaction(
            sender=sender,
            receiver=receiver,
            amount=amount
        )

    return transaction

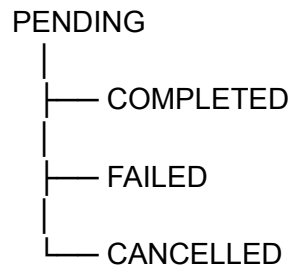
```

The interns don't have to copy this implementation. **They should design their own solution.**

---

## 18. Important State Transitions

Make them implement transaction states correctly.





But don't allow arbitrary transitions.

For example:

PENDING → COMPLETED 

PENDING → FAILED 

PENDING → CANCELLED 

COMPLETED → PENDING 

COMPLETED → FAILED 

FAILED → COMPLETED 

This teaches them to think about **domain state**, rather than just database fields.

---